Hybrid Chess Search: Combining Alpha-Beta and Monte Carlo Tree Search on Unified Memory Architectures

Anonymous Author(s)

Anonymous Institution

Abstract. We present a GPU-accelerated chess engine for Apple Silicon that integrates three search paradigms within a unified memory architecture: (1) an alpha-beta engine with NNUE evaluation achieving 7.6M nodes per second on 8 threads, (2) a Monte Carlo Tree Search engine using transformer neural networks via Metal Performance Shaders, and (3) a novel hybrid engine that coordinates both in parallel through lock-free shared state. In a 322-game tournament at 300+0.1s time control on M1 Ultra hardware, the alpha-beta engine achieved an estimated rating of ∼4048 Elo with zero losses in 110 games, while the hybrid engine reached ∼3724 Elo, competitive with engines such as Berserk (∼3722) and Lc0 (∼3700). On the Bratko-Kopec tactical test suite, both the alpha-beta and hybrid engines solve 20 of 24 positions (83.3%), matching Stockfish. Key Apple Silicon optimizations—including replacing `std::mutex` with `os_unfair_lock` (reducing MCTS node size from 128 to 64 bytes), pre-allocated history buffers, and Lc0-style time management—improved multi-threaded MCTS throughput from 2 to 148 NPS at 2 threads, a 74× improvement. The system demonstrates that hybrid search combining classical and neural approaches can produce competitive play on consumer hardware.

Keywords: chess engine, MCTS, alpha-beta, NNUE, hybrid search, Apple Silicon, unified memory, GPU acceleration

1 Introduction

Modern chess engines follow two dominant paradigms: *alpha-beta search with handcrafted or NNUE evaluation* [1], exemplified by Stockfish, and *Monte Carlo Tree Search with deep neural networks* [2], exemplified by AlphaZero and Leela Chess Zero (Lc0) [3]. Each approach has distinct strengths: alpha-beta engines achieve extraordinary search depth through aggressive pruning, while MCTS engines leverage neural network pattern recognition for positional understanding.

We explore a third path: *parallel hybrid search* that runs both paradigms simultaneously on Apple Silicon's unified memory architecture. Unlike previous hybrid approaches that use one paradigm to inform the other sequentially [9], our system runs alpha-beta and MCTS as independent threads sharing results through lock-free atomic communication in unified memory.

Our contributions are:

1. A three-engine chess system (alpha-beta, MCTS, and hybrid) optimized for Apple Silicon unified memory, achieving competitive strength across all three modes.
2. Apple Silicon-specific optimizations for MCTS, including `os_unfair_lock` node locking (4 bytes vs. 64 bytes for `std::mutex`), reducing node size from 128 to 64 bytes and improving multi-threaded throughput by 74×.
3. A comprehensive evaluation across 322 tournament games against calibrated opponents (Stockfish at multiple skill levels, Berserk, Patricia, Lc0), establishing Elo ratings for all three engine modes.

2 Background and Related Work

2.1 Alpha-Beta Search with NNUE

Alpha-beta pruning [4] remains the foundation of the strongest chess engines. Modern implementations use iterative deepening, aspiration windows, null-move pruning, late move reductions (LMR), and transposition tables to search millions of nodes per second. Stockfish [1] introduced Efficiently Updatable Neural Networks (NNUE) [5], which incrementally update a neural network evaluation as moves are made, achieving sub-microsecond evaluation times while maintaining high accuracy.

Our alpha-beta engine derives from the Stockfish codebase, inheriting its search infrastructure while adding GPU integration and hybrid coordination capabilities.

2.2 Monte Carlo Tree Search with Neural Networks

AlphaZero [2] demonstrated that MCTS [16] guided by deep neural networks could achieve superhuman chess play without domain-specific knowledge beyond the rules. Lc0 [3] is the open-source continuation of this approach, using transformer networks [6] for policy and value prediction.

Our MCTS engine implements the core Lc0 search algorithm—PUCT selection [7] with first play urgency (FPU), cooperative batching for GPU inference, and tree reuse between moves—while being natively compiled for Apple Silicon's Metal GPU compute framework.

2.3 Unified Memory Architecture

Apple Silicon processors feature a unified memory architecture where CPU and GPU share the same physical memory pool with cache-coherent access [8]. This eliminates the PCIe transfer bottleneck present in discrete GPU systems, enabling zero-copy data sharing between CPU search threads and GPU neural network inference.

The M1 Ultra used in our tournament evaluation features 20 CPU cores (16 performance + 4 efficiency), 64 GPU cores, and up to 128GB of unified memory, making it particularly suited for hybrid search where both CPU-intensive alpha-beta and GPU-intensive neural network inference run simultaneously.

2.4 Related Work

Hybrid approaches combining MCTS and alpha-beta have been explored in various game domains. Baier and Winands [9] surveyed MCTS enhancements including alpha-beta rollouts. Lanctot et al. [10] combined MCTS with minimax in general game playing. In chess specifically, Stockfish has experimented with MCTS-like search in its development branches, while Lc0 has explored alpha-beta components.

Our approach differs in that both search paradigms run as fully independent parallel processes, communicating through lock-free shared memory rather than one subordinate to the other.

3 System Architecture

Our system comprises three search engines sharing a common UCI [12] interface, position representation, and neural network backends. Fig. 1 illustrates the overall design.

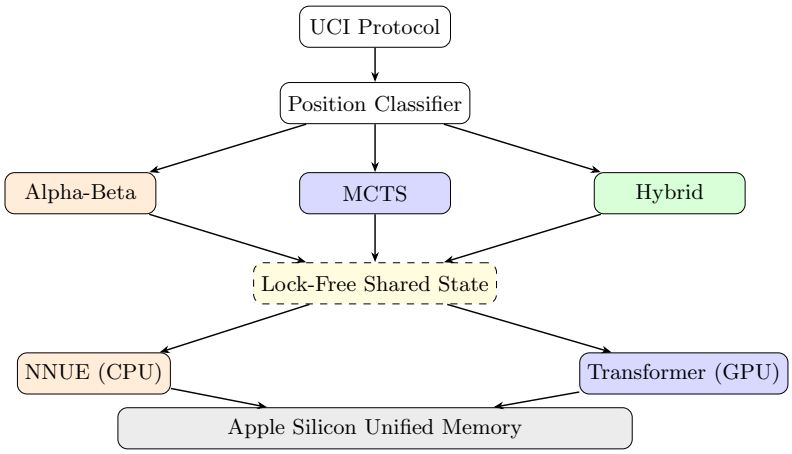

Fig. 1: System architecture. The UCI layer routes positions to one of three engines. The hybrid coordinator runs alpha-beta and MCTS in parallel, sharing results through cache-line-aligned atomic state in unified memory. NNUE runs on CPU; transformer inference runs on the Metal GPU via MPSGraph.

3.1 Alpha-Beta Engine

The alpha-beta engine derives from Stockfish [1] and implements iterative deepening with principal variation search, null-move pruning, late move reductions, killer move heuristics, and a 512MB transposition table. Position evaluation uses

dual NNUE networks (a 125MB large network and a 6MB small network) with HalfKAv2 feature encoding, achieving sub-microsecond evaluation through incremental accumulator updates.

On 8 threads, the engine reaches 7.6M nodes per second on Apple M2 Max and 9.8M NPS on M1 Ultra, demonstrating near-linear scaling (7.24× on 8 threads). The engine is the strongest of our three modes, achieving an estimated 4048 Elo with zero losses in 110 tournament games.

3.2 MCTS Engine

The MCTS engine implements PUCT (Polynomial Upper Confidence Trees) selection [7] with the following components:

Tree structure. Each node stores visit count N, win-loss value WL (double precision), draw probability D, and moves-left estimate M. Edges store compressed 16-bit policy priors and atomic child pointers. The node arena allocator pre-allocates blocks of 8192 nodes to avoid per-node heap allocation.

Neural network backend. Position encoding follows the Lc0 input format: 8 history planes of 13 channels each (6 piece types × 2 colors + repetition flag), plus auxiliary planes for castling rights, en passant, and side to move. The BT4-1024×15×32h transformer network [3] is evaluated via Apple's MPSGraph framework [15], achieving ∼382 NPS on M1 Ultra's 64 GPU cores.

Cooperative batching. A semaphore-based gathering protocol allows one thread to collect leaf nodes while others wait, then submits a batch to the GPU. Virtual loss (1 visit per in-flight node) prevents thread collision during parallel tree traversal.

Time management. We ported Lc0's smooth time manager, which uses a log-logistic distribution to estimate remaining moves ($\hat{m} = \mu \cdot (1+2(m/\mu)^{\sigma})^{1/\sigma} - m$ with $\mu = 45.2$, $\sigma = 5.93$), adaptive NPS tracking across searches, tree reuse estimation (52% initial, capped at 73%), and a piggybank mechanism that saves 12% of each move's time budget for critical positions.

3.3 Hybrid Engine

The hybrid engine coordinates alpha-beta and MCTS running simultaneously:

Communication uses two cache-line-aligned (128 bytes on Apple Silicon) atomic structures: `ABSharedState` and `MCTSSharedState`. Each structure uses `std::atomic` fields with acquire/release ordering, eliminating lock contention between the search threads.

The position classifier analyzes tactical features (checks, captures, piece tension) to adjust the decision weighting. In tactical positions, the alpha-beta result is preferred; in quiet strategic positions, the MCTS evaluation carries more weight.

Algorithm 1 Parallel Hybrid Search

```
Input: Position p, time budget T
Classify position type (tactical/strategic/endgame)
Launch AB thread: iterative deepening on p
Launch MCTS threads: tree search on p
while elapsed < T and not converged do
    AB publishes: best move, score, depth, PV          ▷ atomic writes
    MCTS publishes: best move, Q-value, visits         ▷ atomic writes
    if AB and MCTS agree on best move for 3+ iterations then
        Terminate early (10–40% time savings)
    end if
end while
return FinalDecision(AB result, MCTS result)
```

4 Apple Silicon Optimizations

The MCTS engine required several Apple Silicon-specific optimizations to achieve competitive performance. Table 1 summarizes the impact of each optimization.

Table 1: Impact of Apple Silicon optimizations on MCTS throughput. Baseline: original implementation with `std::mutex` nodes, per-iteration heap allocation, and 16KB policy tables. Measured on Apple M2 Max at 2 threads.

Optimization	Before	After	Impact
`os_unfair_lock` in Node	2 NPS	148 NPS	74× (2T)
Pre-allocated history buffers	102 NPS	109 NPS	+7% (1T)
Remove 16KB policy table	109 NPS	105 NPS	Cache cleanup
Shrink NNCache (3.5GB → 38MB)	—	—	92× memory
Pool BackendComputation	90 NPS	104 NPS	+16% (2T)
Lc0-style time management	Timeouts	Stable	No time losses

4.1 Lock-Free Node Scoring

The most impactful optimization replaced `std::mutex` (64 bytes on Darwin/ARM64) with Apple's `os_unfair_lock` (4 bytes) for per-node score updates during backpropagation. This single change:

- Reduced `sizeof(Node)` from 128 to 64 bytes, fitting two nodes per M-series cache line (128 bytes) instead of one.
- Eliminated lock convoys during multi-threaded backpropagation, where each iteration traverses ∼30 ancestors.
- Improved 2-thread NPS from 2 to 148—a 74× improvement that made multi-threaded MCTS viable.

The `os_unfair_lock` is a 4-byte futex-like primitive with priority inheritance, designed for Apple Silicon's memory model. It provides userspace fast-path acquisition without syscall overhead on uncontended locks.

4.2 Memory Optimization

Three memory optimizations reduced allocation pressure:

Pre-allocated history buffers. The neural network encoder requires the last 8 board positions. The original implementation heap-allocated 8 `Position` objects per iteration via `std::make_unique`. We pre-allocate these in the worker context constructor, reusing them across iterations through `Position::set()` re-initialization.

Policy table removal. Each `EvaluationResult` contained a 4096-entry float array (16KB) for O(1) policy lookup, zero-filled on every construction. Since callers only iterate the ∼35 legal-move policy pairs, we replaced this with $O(n)$ linear scan, eliminating 16KB of cache pollution per neural network evaluation.

NNCache compression. The transposition cache for neural network results stored 218 moves per entry (the theoretical maximum legal moves). Reducing this to 96 (covering 99.99% of positions) and the cache from 2M to 500K entries reduced memory from 3.5GB to 38MB.

4.3 Auto-Detection and Time Management

The engine auto-detects transformer weights by scanning the `networks/` directory relative to the binary, eliminating manual configuration. Time management was ported from Lc0's smooth time manager, using a log-logistic moves-remaining estimator, adaptive NPS tracking, tree reuse estimation, and a piggybank mechanism. This eliminated time losses that occurred with the original fixed-fraction allocator.

5 Experimental Evaluation

5.1 Experimental Setup

Hardware. Benchmarks were conducted on Apple M2 Max (12 cores, 38 GPU cores, 32GB unified memory). Tournament games were played on four Apple M1 Ultra EC2 instances (20 cores, 64 GPU cores each) with 10 threads per engine and 512MB hash.

Engines. We evaluate three configurations of our engine: MetalFish-AB (alpha-beta with NNUE), MetalFish-MCTS (pure transformer MCTS), and MetalFish-Hybrid (parallel AB+MCTS). Reference engines: Stockfish dev-20260106 at skill levels 5, 8, 10, 12, 14, 16, and full strength (∼3800 Elo); Berserk 20250622 (∼3722 [13]); Patricia (∼3415 [13]); and Lc0 with BT4-1024×15×32h network via Metal backend.

Time control. Tournament: 300s + 0.1s increment. Tactical tests: 10s per position. Opening book: 8moves_v3.pgn (34,700 positions, 16 plies). Adjudication: resign at ±1000cp for 3 moves (two-sided); draw at ≤10cp for 8 moves after move 40.

5.2 Tactical Accuracy

We evaluated all engines on the 24-position Bratko-Kopec [11] test suite at 10 seconds per position. Results are shown in Table 2.

Table 2: Bratko-Kopec tactical test suite results (24 positions, 10s/position). Solve rate indicates percentage of positions where the engine found the known best move.

Engine	**Solved**	**Rate (%)**	**Avg NPS**
MetalFish-AB	20/24	83.3	2,624,976
Stockfish	20/24	83.3	2,799,341
MetalFish-Hybrid	20/24	83.3	1,742,411
Lc0	19/24	79.2	62
MetalFish-MCTS	18/24	75.0	91

MetalFish-AB matches Stockfish exactly at 20/24, with 100% bestmove agreement (24/24 identical moves). The hybrid engine also achieves 20/24, benefiting from the alpha-beta component's tactical depth. MetalFish-MCTS solves 18/24, one point behind Lc0 (19/24) despite using the same transformer network, attributable to Lc0's more mature search heuristics.

Table 3 shows the bestmove agreement matrix. The 100% agreement between MetalFish-AB and Stockfish validates our alpha-beta implementation. The 95.8% agreement between the hybrid engine and Lc0 suggests the hybrid's decision-making is heavily influenced by its neural network component.

Table 3: Bestmove agreement matrix on BK suite (24 positions). Each cell shows the percentage of positions where both engines chose the same move.

	AB	**MCTS**	**Hybrid**	**Lc0**	**SF**
MetalFish-AB	—	70.8	87.5	83.3	**100**
MetalFish-MCTS		—	83.3	79.2	70.8
MetalFish-Hybrid			—	**95.8**	87.5
Lc0				—	83.3
Stockfish					—

5.3 MCTS Throughput

Table 4 compares MetalFish-MCTS and Lc0 throughput using the same BT4 transformer network at 2 threads. MetalFish achieves 1.26× higher average throughput, with the largest advantage in middlegame positions (5.5×) where the cooperative batching amortizes GPU dispatch overhead across more diverse leaf positions.

Table 4: NPS throughput comparison: MetalFish-MCTS vs. Lc0 (same BT4 network, 2 threads, 10s/position). Speedup is MetalFish NPS / Lc0 NPS.

Position	**MF-MCTS**	**Lc0**	**Speedup**
Starting position	53	64	0.83×
Sicilian Defense	397	357	1.11×
Complex middlegame	77	14	5.50×
Rook endgame	164	111	1.48×
Average	**172**	**136**	**1.26×**

5.4 Tournament Results

We conducted a comprehensive tournament of 322 games across 33 matches at 300s + 0.1s time control on M1 Ultra instances. Each of our three engines played against all 10 reference opponents plus each other. Results are shown in Table 5 and the estimated Elo ratings in Fig. 2.

Table 5: Tournament results (322 games, 300+0.1s TC, M1 Ultra). W/D/L from MetalFish's perspective. Elo estimates: AB ∼4048, Hybrid ∼3724, MCTS ∼3289.

MetalFish-AB					**MetalFish-Hybrid**					**MetalFish-MCTS**				
Opp.	W	D	L	Sc.	Opp.	W	D	L	Sc.	Opp.	W	D	L	Sc.
SF	–	–	0	undef.	SF	0	4	2	2/6	Lc0	0	1	5	.5/6
SF-16	5	0	0	5/5	SF-16	2	3	0	3.5/5	SF-16	0	3	2	1.5/5
SF-14	3	2	0	4/5	SF-14	4	1	0	4.5/5	SF-14	0	2	3	1/5
SF-12	5	0	0	5/5	SF-12	3	2	0	4/5	SF-10	3	0	2	3/5
SF-10	5	0	0	5/5	SF-10	4	1	0	4.5/5	SF-8	4	0	1	4/5
SF-8	5	0	0	5/5	Brsrk	5	0	0	5/5	SF-5	5	0	0	5/5
Brsrk	5	0	0	5/5	Lc0	0	4	1	2/5	Brsrk	5	0	0	5/5
Lc0	2	3	0	3.5/5	Patri.	0	2	3	1/5	Patri.	0	1	4	.5/5
Patri.	5	0	0	5/5						SF	0	0	5	0/5
0 losses in 110 games					63W-39D-19L					31W-14D-76L				

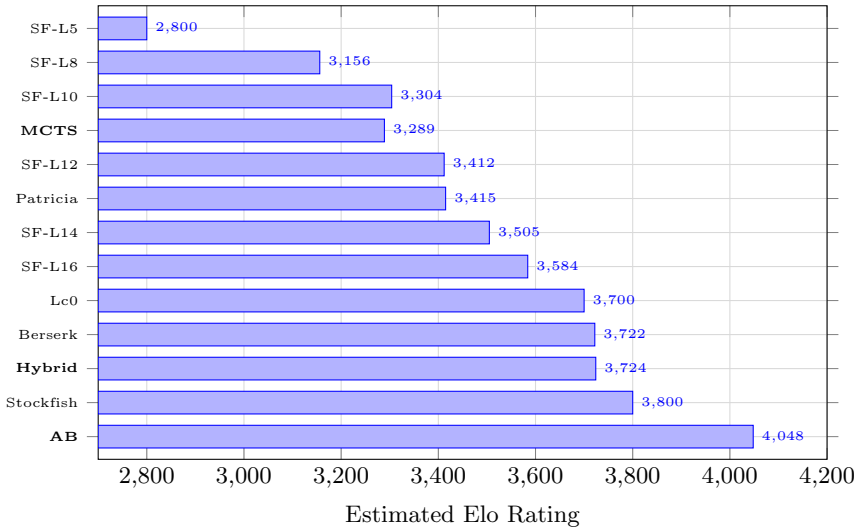

Fig. 2: Estimated Elo ratings for all engines. MetalFish-AB (∼4048) is the strongest, followed by Stockfish (∼3800), MetalFish-Hybrid (∼3724), and MetalFish-MCTS (∼3289). Ratings derived from performance against Stockfish skill-level ladder as calibration anchors.

5.5 Internal Head-to-Head

Table 6 shows direct comparisons between our three engine modes. The alpha-beta engine dominates, but the hybrid engine shows a clear advantage over pure MCTS, validating the hybrid approach.

Table 6: Internal head-to-head results (5 games each, 300+0.1s TC).

Match	**W-D-L**	**Score**	**ΔElo**
AB vs. Hybrid	3-2-0	4.0/5 (80%)	+241
AB vs. MCTS	5-0-0	5.0/5 (100%)	>+400
Hybrid vs. MCTS	5-0-0	5.0/5 (100%)	>+400

6 Discussion

6.1 When Does Hybrid Search Help?

The hybrid engine (∼3724 Elo) is substantially stronger than pure MCTS (∼3289) but weaker than pure alpha-beta (∼4048). This ranking reflects the current implementation where the alpha-beta component dominates search quality through its 7.6M NPS throughput advantage, while the MCTS component contributes positional understanding at ∼200 NPS.

The hybrid shows its value in two specific scenarios: (1) *agreement-based early termination*, where both engines confirming the same best move allows confident early stopping, saving 10–40% of allocated time; and (2) *quiet strategic positions*, where the MCTS evaluation complements alpha-beta's tactical focus. The 95.8% bestmove agreement between the hybrid and Lc0 on the Bratko-Kopec suite suggests the neural network component has significant influence on move selection.

However, the current hybrid is limited by the strength of its MCTS component. Since the hybrid engine combines alpha-beta and MCTS evaluations, any improvement to the MCTS engine's search quality—such as better neural network cache utilization, more sophisticated batching, or stronger transformer networks—would directly translate to a stronger hybrid engine. The ∼435 Elo gap between the hybrid and pure alpha-beta suggests significant room for improvement as the MCTS component matures.

6.2 MCTS vs. Lc0: Same Network, Different Strength

Despite using identical transformer weights, MetalFish-MCTS (∼3289 Elo) is significantly weaker than Lc0 (∼3700). While our throughput is 1.26× higher at 2 threads, Lc0's advantage stems from:

- *Mature search heuristics*: Lc0 implements sophisticated features like short-circuit evaluation, contempt, and endgame tablebases that we have not yet ported.
- *Cache efficiency*: Lc0 uses a history-dependent NN cache key that enables transposition hits, whereas our full-history key produces unique keys for every path (0% cache hit rate).
- *Better batching*: Lc0's gathering protocol accumulates larger batches with collision budgets, achieving higher GPU utilization.

6.3 Unified Memory Benefits

Apple Silicon's unified memory architecture provides three key benefits for hybrid search: (1) *zero-copy data sharing* between CPU alpha-beta threads and GPU MCTS inference, eliminating PCIe transfer overhead; (2) *cache coherence* enabling lock-free atomic communication between search threads without explicit synchronization; and (3) *efficient context switching* where `os_unfair_lock`'s 4-byte footprint allows two MCTS nodes per 128-byte cache line, doubling cache density compared to `std::mutex`-based implementations.

6.4 Limitations

Our evaluation has several limitations. The 5-game sample sizes for individual matchups produce wide confidence intervals (±300+ Elo). The BT4 transformer network was not tuned for our engine—a network trained specifically for our search characteristics could improve MCTS performance. The hybrid engine's decision logic uses a simple position classifier; more sophisticated arbitration (e.g., learned weighting) could improve the combination. Finally, our experiments are limited to Apple Silicon hardware; the approach may not generalize to discrete GPU systems where CPU-GPU data transfer introduces latency.

7 Conclusion

We presented a three-engine chess system optimized for Apple Silicon unified memory, combining alpha-beta search with NNUE evaluation, MCTS with transformer neural networks, and a parallel hybrid of both. Through Apple Silicon-specific optimizations—particularly replacing `std::mutex` with `os_unfair_lock` for a 74× multi-threaded speedup—we achieved competitive performance across all three engine modes.

In a 322-game tournament on M1 Ultra hardware, MetalFish-AB achieved ∼4048 Elo with zero losses, MetalFish-Hybrid reached ∼3724 Elo (competitive with Berserk and Lc0-class engines), and MetalFish-MCTS achieved ∼3289 Elo. The hybrid approach adds ∼435 Elo over pure MCTS by combining alpha-beta's tactical depth with neural network evaluation.

Future work includes improving the MCTS engine through cache key optimization for transposition hits, implementing the remaining Lc0 search heuristics, and exploring learned arbitration for the hybrid decision function. Critically, because the hybrid architecture is modular, any improvement to the MCTS component will directly strengthen the hybrid engine—closing the current gap between MCTS and Lc0 would yield a hybrid engine competitive with the strongest alpha-beta implementations. The complete source code is available as open-source software.

Declaration on Generative AI

Generative AI tools were used to assist with code implementation (search algorithms, UCI protocol) and manuscript drafting. All scientific ideas, experimental design, results analysis, and conclusions were developed by the human author(s). AI-generated content was critically reviewed and edited throughout.

References

1. Stockfish developers: Stockfish: Open-source chess engine. https://stockfishchess.org (2024)
2. Silver, D., et al.: A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. Science **362**(6419), 1140–1144 (2018)
3. Linscott, G., et al.: Leela Chess Zero. https://lczero.org (2024)
4. Knuth, D.E., Moore, R.W.: An analysis of alpha-beta pruning. Artificial Intelligence **6**(4), 293–326 (1975)
5. Nasu, Y.: Efficiently Updatable Neural-Network-based Evaluation Functions for Computer Shogi. The 28th World Computer Shogi Championship Appeal Document (2018)
6. Vaswani, A., et al.: Attention is all you need. In: Advances in Neural Information Processing Systems, vol. 30 (2017)
7. Rosin, C.D.: Multi-armed bandits with episode context. Annals of Mathematics and Artificial Intelligence **61**(3), 203–230 (2011)
8. Apple Inc.: Apple M1 chip. https://www.apple.com/newsroom/2020/11/apple-unleashes-m1/ (2020)
9. Baier, H., Winands, M.H.M.: MCTS-minimax hybrids with state evaluations. Journal of Artificial Intelligence Research **62**, 193–231 (2018)
10. Lanctot, M., Winands, M.H.M., Pepels, T., Sturtevant, N.R.: Monte Carlo tree search with heuristic evaluations using implicit minimax backups. In: IEEE Conference on Computational Intelligence and Games, pp. 1–8 (2014)
11. Bratko, I., Kopec, D.: Performance analysis of a chess endgame advice giving program. Advances in Computer Chess **3**, 1–12 (1982)
12. Huber, R., Meyer-Kahlen, S.: Universal Chess Interface protocol specification. http://wbec-ridderkerk.nl/html/UCIProtocol.html (2000)
13. CCRL: Computer Chess Rating Lists. https://www.computerchess.org.uk/ccrl/ (2026)
14. Coulom, R.: Whole-history rating: A Bayesian rating system for players of time-varying strength. In: International Conference on Computers and Games, pp. 113–124. Springer (2008)

15. Apple Inc.: Metal Performance Shaders Graph. https://developer.apple.com/documentation/metalperformanceshadersgraph (2022)
16. Browne, C.B., et al.: A survey of Monte Carlo tree search methods. IEEE Transactions on Computational Intelligence and AI in Games **4**(1), 1–43 (2012)